LOCAL RESEARCH / AUDITABLE BACKTESTING

QuantLab

US Equity Research System

Project, User and Maintenance Manual

English edition

From a trading idea to an auditable historical experiment: data, factors, strategies, execution, accounting, analytics, reports and a desktop interface.

Project version 2026.10.22

Verified against the local checkout on 22 October 2026

Workspace: `/Users/yusa/ChatGPTWorkSpace/QuantLab`

This manual distinguishes implemented capabilities, standalone research scripts and possible extensions. Historical backtests do not establish future profitability.

Contents

1 Purpose, audience and scope **2**
1.1 Implemented capabilities . . . 2
1.2 What the system does not provide . . . 2
1.3 How to read this manual . . . 2

2 Architecture and directory guide **3**
2.1 The path of a backtest . . . 3
2.2 Top-level directories . . . 3
2.3 Core modules and root files . . . 4

3 Installation and the first run **4**
3.1 Prepare the Python environment . . . 4
3.2 An offline smoke test . . . 5
3.3 Use the existing historical dataset . . . 5

4 Desktop application workflow **6**
4.1 Open and connect . . . 6
4.2 Select, edit, configure and run . . . 6
4.3 Distinguish source, form values and results . . . 6
4.4 Development and packaging . . . 7

5 Turn an idea into an executable experiment **7**
5.1 Specify the rule before implementation . . . 7
5.2 The strategy interface . . . 7
5.3 Available observations . . . 8
5.4 Bind configuration and run . . . 9

6 Factors and the existing strategies **9**
6.1 A factor is not a trading policy . . . 9
6.2 The current return-spread factor . . . 9
6.3 Distinguish the existing rules . . . 10

7 Data, configuration and execution semantics **10**
7.1 Data contract and adjustment . . . 10
7.2 Configuration map . . . 11
7.3 Daily event order and weekly timing . . . 11
7.4 Two execution-sizing modes . . . 12
7.5 Costs and missing data . . . 12

8 Interpret performance correctly **12**

9 Archives, reproduction and independent audit **13**
9.1 What a run preserves . . . 13
9.2 Rerun and migrate . . . 14
9.3 The existing independent audit . . . 14

10 Using the standalone research studies **14**

11 Maintenance, verification and delivery **15**
11.1 A disciplined change process . . . 15
11.2 Routine verification commands . . . 15
11.3 Dependencies, backups and publication . . . 15
11.4 Maintaining this manual . . . 16

12 Troubleshooting and operating checklist **16**

A Source basis and version interpretation **17**

1 Purpose, audience and scope

QuantLab is a local research and backtesting system for daily US equity data. Given market data, a Python strategy and an experiment configuration, it simulates historical trading in a defined event order, maintains cash and positions, calculates performance, and preserves an auditable experiment archive. Correctness, auditability and reproducibility take precedence over automatically finding the most profitable rule.

The project has three layers. The Python engine performs the calculations. The command-line interface and Tauri desktop application provide two ways to operate it. The `researchProject/` directory holds separate research studies and their calculation materials. The UI does not replace engine semantics, and a research report is not automatically an executable item in the strategy menu.

1.1 Implemented capabilities

- Read local daily CSV/Parquet data and validate prices, symbols, sessions and benchmark coverage.
- Execute user-written Python strategies against history truncated at the decision time, with execution at the next session's open.
- Model a long-only USD portfolio with integer units, commissions and slippage; record orders, fills, positions and equity.
- Produce return, risk, cost, turnover and benchmark analytics, plus a self-contained offline HTML report.
- Freeze the configuration, strategy, bound factors, actual data input and engine source for later examination and reproduction.
- Select, edit and save strategy/factor files in the desktop app; configure, validate, run or cancel experiments and inspect archived results.

1.2 What the system does not provide

There is no broker connection or live order submission. The core does not support shorting, leverage, intraday bars, order books, limit orders or delisting liquidation. The universe is a fixed list rather than a point-in-time constituent database. Installed core workflows and tests can run offline; installing dependencies or explicitly using the Yahoo adapter requires network access or prepared caches.

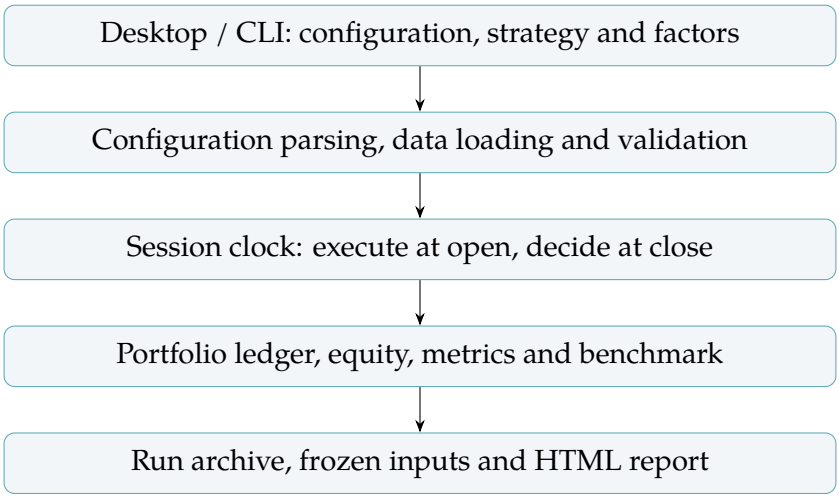
The desktop app uses `.venv/bin/python` inside the selected workspace. It does not bundle a complete Python runtime, so copying only the App to another computer is not a complete installation. Strategy checks and restricted data interfaces enforce contracts for trusted code; they are not an adversarial Python security sandbox.

1.3 How to read this manual

Start with Sections 3 and 4 for installation and daily use. Sections 5 and 6 explain strategy/factor development. Sections 7 and 8 explain execution and results. Sections 9–12 cover reproduction, research materials, maintenance and troubleshooting. Shell examples assume the project root is the working directory and its virtual environment is activated unless stated otherwise.

2 Architecture and directory guide

2.1 The path of a backtest



The strategy specifies desired holdings. The engine determines execution time, price, quantities and costs. A factor only computes a feature from available history. This separation makes a change in the investment rule distinguishable from a change in the execution model.

2.2 Top-level directories

The table includes business directories and generated artifacts found in the current checkout. Finder may hide names beginning with a dot.

Directory	Purpose and maintenance considerations
`src/quantlab/`	Installable Python core: data, engine, analytics, reports and application entry points.
`strategies/`	User strategies and an inert template. Preserve the rule specification and version when editing.
`factors/`	Approved pure feature calculations. Dependencies are explicitly bound, not automatically executed as a library of signals.
`configs/`	YAML experiment definitions; `examples/` contains the basic example. Separate configurations preserve old conditions.
`data/raw/`	Local original/frozen input, including synthetic sample data and historical market Parquet files. Keep their provenance distinct.
`data/processed/`, `data/cache/`, `data/metadata/`	Organizational locations for transformed data, caches and provenance. Actual use depends on the configuration and script; every run need not use every directory.
`runs/`	Independent experiment archives. These are evidence for analysis and reproduction, not disposable caches.
`desktop/`	React/TypeScript UI and Tauri shell. Its `src/` holds frontend code and `src-tauri/` holds Rust.

Directory	Purpose and maintenance considerations
`scripts/`	Sample generation, data validation, clean-install checks and independent arithmetic auditing.
`tests/`	Offline automated tests, including small hand-checkable data in `fixtures/`.
`researchProject/`	Standalone studies with code, data, results and reports. Do not assume identical semantics to the core engine.
`validation/`	Earlier acceptance and research logs. Historical evidence is not a new verification of today's code.
`dist/`, `build/`	App/wheel deliverables and build intermediates. Source changes require rebuilding the affected deliverable.
`.quantlab-desktop/`	Desktop job snapshots and logs. Do not clean it during a run; cancellation may leave intermediate files.
`.venv/`, `.wheelhouse/`	Python environment and locally cached dependency wheels. A virtual environment is generally not portable by copying it.
`.git/`	Version history and remote configuration, not research data.
`.pytest_cache/`, `.ruff_cache/`	Rebuildable testing/tool caches, distinct from archived experiments.

2.3 Core modules and root files

Within `src/quantlab/`, `data/` handles calendars, input checks and truncated views; `strategy/` defines contracts, context, loading and validation; `engine/` owns the clock, execution, orders and ledger; `costs/` implements commission and slippage; `metrics/` defines statistical conventions; `report/` builds charts and HTML; and `utils/` supports serialization, paths and logging.

`config.py` parses strict configurations, `cli.py` exposes commands, `archive.py` creates run archives, and `desktop_bridge.py` provides UI file, configuration, job and result operations. `types.py` contains shared types/exceptions; package initialization files provide Python package entry points.

At the root, `pyproject.toml` defines package metadata and dependency ranges; `requirements.lock` pins the accepted environment; `README.md` provides quick instructions; and `AGENTS.md` records development constraints and approved strategy rules. The V4 work document is the original specification. Acceptance documents and `DESKTOP_PROGRESS.md` are historical records. Resolve discrepancies by inspecting current code, tests and the specific archived experiment.

3 Installation and the first run

3.1 Prepare the Python environment

The package declares Python 3.12 or newer. First inspect an existing environment rather than recreating it unnecessarily:

```
cd /Users/yusa/ChatGPTWorkSpace/QuantLab
source .venv/bin/activate
python --version
python -m pip check
quantlab --help
```

For a new environment, select the intended Python interpreter and then run:

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.lock
python -m pip install -e '.[dev]'
python -m pytest -q
```

Editable installation makes changes under `src/` effective in that environment. A dependency lock file is not a cross-platform installer: offline installation also needs compatible wheels. The current virtual environment may reference an external Python runtime. If that runtime is removed or moved, rebuild the environment rather than only changing its activation script.

3.2 An offline smoke test

```
python scripts/make_sample_data.py
python scripts/validate_data.py configs/default.yaml
quantlab strategies validate \
  strategies/fixed_weight_buy_and_hold.py \
  --config configs/examples/fixed_weight_buy_and_hold.yaml
quantlab backtest run \
  --config configs/examples/fixed_weight_buy_and_hold.yaml
```

The generator creates synthetic prices with a fixed seed. Symbols such as AAPL, TSLA and SPY do not make those generated prices real historical observations. Use this example to verify installation and the complete pipeline, not to draw investment conclusions.

The command prints the run directory and final equity. Replace `RUN_ID` below with the actual identifier:

```
quantlab report open RUN_ID
```

When working outside the project root, use `--runs-dir/absolute/path/to/runs`. Alternatively, open the archive's `report.html` directly in a browser.

3.3 Use the existing historical dataset

Real market data are not distributed with this release. In a fresh clone, first generate the synthetic sample and select its basic configuration in the App. The historical experiments below require locally prepared data at the configured paths.

The existing weekly research configurations point to a local Yahoo historical Parquet file. Running them does not download fresh data. Validate and run as follows:

```
python scripts/validate_data.py \
  configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
quantlab strategies validate \
  strategies/weekly_equal_weight_aapl_tsla.py \
  --config configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
quantlab backtest run \
  --config configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
```

This configuration begins with a cash baseline on 3 January 2020 so the first instruction can execute at the open on 6 January. Moving the configured start directly to the intended purchase day changes available history, the first execution and the performance baseline.

4 Desktop application workflow

4.1 Open and connect

Open `dist/QuantLab.app` in Finder. If the workspace is not detected, enter the project root's absolute path in Workspace Settings. It should contain `pyproject.toml`, `src/quantlab/`, strategies, configurations and a working `.venv/bin/python`. The packaged App does not require the browser development server.

The sidebar provides the research workbench, run history, data catalog and workspace settings. Dark/light theme selection changes appearance only. The data catalog primarily lists local files; it is not a full data procurement, online refresh or cleaning platform.

4.2 Select, edit, configure and run

1. Select a configuration preset close to the intended experiment. Inspect the strategy, universe, benchmark and dates.
2. Select a Python strategy from `strategies/`. Read its actual rule and recheck parameter compatibility after switching files.
3. Select a factor to open it in the editor. If the strategy imports it, also bind the dependency and call its function in code. Opening a factor alone does not create trades.
4. Edit the source and save with the button or Command+S. Multiple documents can remain as tabs. A new strategy starts from the existing template; a new factor is an unimplemented stub.
5. Set dates, capital, universe, benchmark and local data. Expand advanced settings for JSON parameters, sizing and trading costs.
6. Validate the strategy, then run. The application saves edits, validates the configuration and freezes the source used for that job. Inspect logs or cancel when needed.
7. Confirm the result's run identifier, then inspect equity, metrics, fills and the full report.

4.3 Distinguish source, form values and results

Saving code updates local Python files. Editing the form does not automatically overwrite the original YAML configuration; the experiment values are preserved in the run snapshot. Create or edit a YAML file under `configs/` to keep a reusable preset.

An old curve does not mean the current edits have been tested. Only a new successful run corresponds to the new inputs. The history view returns at most the latest 100 entries, and the fill table displays at most 2,000 rows. Older archives and complete fills remain on disk. Damaged entries are isolated as failures, failure markers take precedence, and successful records require complete metadata.

Content fingerprints prevent silently overwriting a file changed by another editor. Copy any draft you need to keep, reload the disk version and merge intentionally. Cancellation does not make partial output a valid result; a leftover directory alone is not evidence of a successful run.

4.4 Development and packaging

Using an existing App does not require Node or Rust. Development/rebuilding requires a Node.js version supported by Vite 6, npm, Rust/Cargo and macOS developer tools. This release updates Vite to the compatible 6.4.5 patched line and source-map-js to 1.2.2; the release-time npm audit reported zero known vulnerabilities:

```
cd desktop
npm ci
npm run desktop
# Build the production macOS app:
npm run bundle
```

For a browser-only development preview:

```
QUANTLAB_WORKSPACE=/Users/yusa/ChatGPTWorkSpace/QuantLab npm run dev
```

The preview binds to `127.0.0.1:1420`. The native application calls Python through Tauri IPC. The new bundle is produced at `desktop/src-tauri/target/release/bundle/macos/QuantLab.app`. Test it, quit the old App and then replace the delivered copy. Editing source does not update an already packaged binary.

5 Turn an idea into an executable experiment

5.1 Specify the rule before implementation

A verbal request should define the tradable symbols and observable benchmark; exact arithmetic and thresholds; initial allocation; signal interval; decision and execution times; holiday handling; cash and position sizing; costs; start/end dates; missing-data behavior; parameters; and any prohibition on automatic optimization.

For example, "equal weight every Monday" is ambiguous between Monday-open execution and Monday-close decisions for Tuesday execution. The approved AAPL/TSLA strategy decides at the final close of the preceding trading week and executes at the first open of the next week.

A typical implementation delivers `strategies/name.py` and `configs/name.yaml`, plus `factors/name.py` only when a new feature is needed. An engine change must be identified and tested separately. Neither a developer nor an AI assistant should silently change the rule to improve returns. The workflow is specification, implementation, validation, backtest, audit and reporting.

5.2 The strategy interface

Copy `strategies/_template.py`, or create a new strategy in the App. This minimal inert skeleton is an interface example, not an additional investment rule:

```
from quantlab.strategy.base import BaseStrategy

STRATEGY_META = {
    "name": "my_strategy", "version": "0.1.0",
    "description": "Document the exact approved rule",
}

class Strategy(BaseStrategy):
    def warmup_bars(self):
```

```
        return 0

    def generate_target_weights(self, dt, data, ctx):
        return None

    def decision_metadata(self):
        return {}

def build_strategy(params):
    return Strategy(params)
```

`build_strategy(params)` creates the instance. The base class copies parameters, and the strategy may keep its own historical state. `decision_metadata()` supplies auditable explanations such as factor values and trigger reasons; it must not submit extra orders.

Return values have distinct meanings: `None` means no new instruction; an empty dictionary means an all-cash target; a nonempty dictionary specifies the whole target portfolio, so omitted existing holdings have a zero target. Weights must be finite, nonnegative, within the tradable universe and inside position/gross-exposure limits. A sum below one leaves a cash target. Subsequent market drift does not trigger automatic enforcement of the target weights.

5.3 Available observations

Interface	Meaning
`data.history(symbol,bars=None,end=None)`	A copy of history visible at the decision cutoff, optionally limited by count and end date.
`data.current(symbol)`	The current visible bar or `None`; missing data must be handled.
`data.as_of`	The view's session cutoff.
`ctx.cash`, `ctx.equity`	Read-only current cash and equity.
`ctx.positions`, `ctx.universe`	Read-only quantity mapping and permitted tradable symbols.
`ctx.next_session_date`	The next session from known calendar metadata, not a future market observation.
`ctx.seed`, `ctx.execution_sizing`	Random seed and active execution-sizing mode.

Benchmark history is observable without making the benchmark tradable. Warmup counts only observable bars inside the chosen backtest interval. The engine does not automatically load training history before that interval; configure enough history for the rule.

Strategies must not download data, access future information, import private engine objects or mutate the ledger. Validation checks common prohibited imports and file I/O, but it proves neither arbitrary Python safety nor investment efficacy.

5.4 Bind configuration and run

Copy an existing YAML file and edit `strategy.path`, `params`, the universe and data path. YAML-relative paths resolve from the YAML file's directory. An explicit CLI `--strategy` path resolves from the current shell directory and overrides the YAML strategy.

```
quantlab strategies validate strategies/my_strategy.py \
  --config configs/my_strategy.yaml
quantlab backtest run --config configs/my_strategy.yaml
```

Passing validation establishes a basic contract only. Inspect edge cases, key decision dates, holdings and cash, and check that the observed output matches the written rule.

6 Factors and the existing strategies

6.1 A factor is not a trading policy

A factor is a calculated feature. A strategy uses features, timing and portfolio state to decide trades. For example, the daily AAPL-minus-SPY return spread is

$$f_t = \left(\frac{P_t^A}{P_{t-1}^A} - 1\right) - \left(\frac{P_t^M}{P_{t-1}^M} - 1\right).$$

Prices here use the system's adjusted series. This is neither a beta-adjusted residual nor direct evidence of overvaluation/undervaluation. A threshold of ± 0.01 means a difference of one percentage point, not an additional percentage scaling of the spread.

The factor determines neither trade size nor execution time. Different strategies can use the same feature. Conversely, restoring fixed 50/50 weights weekly needs no predictive factor, so it does not inherently create a factor-library file.

6.2 The current return-spread factor

`factors/return_spread.py` provides `daily_return_spread` and `weekly_return_spread`. The daily function uses two synchronized daily closes. The weekly function compares the final closes of two adjacent completed trading weeks. A weekly return is neither the last daily return nor simply the arithmetic sum of daily returns.

Call the weekly function only after the strategy has established that the current close is the last session of the week; otherwise a partial week could be treated as complete. Insufficient history returns `None`; unsynchronized endpoints and nonpositive prices fail explicitly. The result includes endpoint dates, asset/benchmark returns and their spread for auditing.

Example binding in a YAML file under `configs/`:

```
strategy:
  path: ../strategies/aapl_weekly_relative_reversal.py
  params:
    start_date: '2020-01-06'
    end_date: '2025-12-31'
    threshold: 0.01
  rebalance_schedule: daily
  factor_paths:
    factors.return_spread: ../factors/return_spread.py
execution:
```

```
  sizing: decision_relative
```

This is a fragment, not a replacement for the other sections of the full configuration. The strategy imports `weekly_return_spread` from `factors.return_spread`. The loader explicitly binds, checks and freezes the module's source. It does not execute every file found in the factor directory.

6.3 Distinguish the existing rules

File / experiment	Implemented behavior
`fixed_weight_buy_and_hold.py`	Submit the initial configured allocation, then do not actively rebalance. The basic example is 50/50 AAPL/TSLA; the half-AAPL control reuses this implementation.
`weekly_equal_weight_aapl_tsla.py`	Begin on 6 January 2020 and restore 50/50 targets at each week's first session open; no factor needed.
`aapl_weekly_relative_reversal.py`	Start with half the cash invested in AAPL. Later weekly spread below -1pp buys with half remaining cash; above +1pp sells half the shares; otherwise hold.
`_template.py`	Inert starting point returning `None`, not a validated investment rule.

The reversal strategy gives initial entry precedence, without a simultaneous factor trade. Cash earns no interest; purchase budgets include commissions; sales round down. Equality at the threshold, with a 10^{-12} boundary tolerance in code, produces no trade. The rule requires `decision_relative`; ordinary target-weight execution is not interchangeable with it.

7 Data, configuration and execution semantics

7.1 Data contract and adjustment

Local input is long format, one symbol/session per row:

```
date,symbol,open,high,low,close,volume,adj_close
```

The date is a timezone-naive session label; decision and fill timestamps use UTC. Data must cover the tradable universe and benchmark under the XNYS calendar. Duplicate symbol/date pairs, invalid prices, negative volume, non-session dates and absent required symbols fail explicitly.

In `adjusted` mode, all OHLC values are scaled by $a_t = \text{adj_close}_t/\text{close}_t$. Dividends and splits are not credited again, and volume retains its source value. Integer quantities are adjusted research units, not literal historical raw shares. `raw` mode does not process corporate actions and rejects explicitly nonzero dividends or nontrivial split indicators. Missing action disclosures can still bias a raw-price study.

When refreshing data, preserve the old file and create a new version with source, download date, interval, adjustment policy and validation results. Vendors can revise adjusted history; frozen inputs are necessary to explain result changes. The optional Yahoo adapter is a convenience source,

not a unique authority. Install `'.[online]'` and explicitly choose `data.provider:yahoo` when required. The desktop workflow primarily uses local files.

7.2 Configuration map

Section	Controls
`backtest`	Start/end dates, positive initial capital and USD base currency.
`data`	Provider, path, daily frequency, adjustment mode, benchmark, provenance and extreme-return warning threshold.
`universe`	A nonempty unique symbol list and fixed-list/current-constituent provenance.
`strategy`	Source, parameters, decision-eligibility schedule and factor module-to-source bindings.
`execution`	Close decisions, next-open execution, integer quantities and sizing mode.
`commission`, `slippage`	Fee model, minimum fee and basis-point slippage. One basis point is 0.01%.
`portfolio`	No shorting or negative cash; single-position and gross-exposure limits.
`analytics`	Annualization sessions (default 252), risk-free rate and rolling window.
`report`	Report generation, automatic opening and output root.
`reproducibility`	Random seed; a seed alone does not replace data/code snapshots.

Strict Pydantic models reject unknown fields and nonfinite values. Duplicate YAML keys also fail. Relative paths are based on the YAML location, not a UI navigation location.

7.3 Daily event order and weekly timing

1. At the open, process the target produced at the preceding session's close.
2. Mark existing holdings at current opens and freeze equity for target conversion.
3. Apply slippage and integer sizing; sell first, then buy in alphabetical symbol order.
4. Deterministically reduce unaffordable buys and cancel the remainder. No borrowing is allowed, and zero fills incur no minimum commission.
5. At the close, value the portfolio, create a truncated history view, invoke the strategy and record the end-of-day state.

`rebalance_schedule:weekly` means decision eligibility at the first session's close each week, not Monday-open execution. The existing Monday-open strategies use `daily` to check whether the next session enters a new week and emit targets only at that boundary. Known calendar metadata is available in advance; future prices are not.

7.4 Two execution-sizing modes

`target_weights` converts desired weights into integer quantities using opening equity. A 50/50 target need not produce exact equal values because of costs, slippage, rounding and cash constraints.

`decision_relative` freezes a cash/share fraction implied by the change in close-time weights, then executes that fraction at the next open. This preserves "spend half the remaining cash" across an overnight gap. It currently supports one tradable symbol, as required by the AAPL reversal, rather than being a general multi-asset sizing algorithm.

7.5 Costs and missing data

Commissions support zero, per-share with a minimum, and notional basis points. Slippage supports zero or basis points, increasing purchase prices and reducing sale prices. It is already embedded in fill prices and must not be charged to cash again. Purchase commissions enter average cost; net sale proceeds determine realized P&L.

Missing execution bars, invalid opens or zero volume cancel orders without future-price substitution. A missing held-position bar uses the last known mark with a warning; a missing benchmark bar fails the run. Targets still pending on the final day are cancelled, but terminal holdings are not forcibly liquidated.

Final daily volume serves only as a zero-volume tradability flag. It is not known at the real open, so this is an explicit daily-model assumption, not an intraday liquidity simulation. The ledger checks

$$E_t = C_t + MV_t = E_0 + \text{RealizedPnL}_t + \text{UnrealizedPnL}_t.$$

8 Interpret performance correctly

Before examining returns, verify provenance, dates, first purchase, benchmark, costs and warnings. A smaller drawdown can result from lower equity exposure rather than better timing. Compare the reversal with the matching initial-half-AAPL buy-and-hold control as well as observing SPY.

Let E_t be closing equity and $r_t = E_t/E_{t-1} - 1$. The first session close is a cash baseline; return statistics omit its placeholder zero. Principal definitions are

$$\begin{aligned}
R_{\text{total}} &= E_T/E_0 - 1, \\
\text{CAGR} &= (E_T/E_0)^{1/Y} - 1, \quad Y = \frac{d_T - d_0}{365.25}, \\
\sigma_{\text{ann}} &= s(r_t)\sqrt{252}, \\
\text{Sharpe} &= \frac{\overline{r_t - r_f}}{s(r_t - r_f)}\sqrt{252}, \\
\text{MDD} &= \min_t \left(\frac{E_t}{\max_{u \le t} E_u} - 1 \right).
\end{aligned}$$

The annualization count is configurable; s is the sample standard deviation and the annual risk-free rate is converted geometrically to a daily rate. Sortino uses the square root of the mean squared negative excess return over all observations. Calmar divides CAGR by absolute maximum drawdown. Drawdown duration uses calendar days, ending an unrecovered episode at the final observation.

Turnover uses two-sided filled notional divided by previous-close equity, then annualizes by the number of return intervals; there is no division by two. Average exposure and cash include the

initial cash-only session. Beta, alpha, tracking error and information ratio use aligned benchmark daily returns. The benchmark is a costless, fractional-share buy-and-hold curve from the first session close, not an executable copy of the strategy's account.

Undefined ratios are `null`/N/A, not zero. Fill count is not completed round-trip count, and the system does not invent a round-trip win rate for buy and hold. HTML reports embed their images for offline reading. Use CSV/Parquet/JSON records, not chart pixels, for numerical audits.

9 Archives, reproduction and independent audit

9.1 What a run preserves

Run directory names include UTC time, strategy name and a digest; new runs do not overwrite earlier experiments. The following files establish what actually ran:

Artifact	Role
`config.snapshot.yaml`	Resolved configuration redirected to frozen inputs.
`config.original.json`	Initially resolved configuration, including original paths.
`strategy.snapshot.py`, `factors.snapshot/`	Strategy and explicitly bound factor sources.
`data.snapshot.parquet`	Actual pre-adjustment input panel, avoiding fresh downloads.
`code.snapshot/`, `requirements.snapshot.txt`	Engine source, installation description and dependency versions.
`metadata.json`, `manifest.json`	Environment, Git state, fingerprints and file SHA-256 values. The manifest excludes itself and the still-written log.
`equity.csv`, `returns.csv`, `benchmark.csv`	Equity, returns and benchmark records.
`decisions.csv`, `orders.csv`, `fills.csv`	Decision explanations, requested/cancelled/executed orders and costs.
`positions.parquet`	Daily holdings.
`summary.json`, `metrics.json`, `warnings.json`	Summary, complete metrics and warnings.
`report.html`, `logs.txt`, `reproduce.sh`	Human-readable report, logs and reproduction entry point.
`failure.json`	Failure marker; the directory is not a successful experiment.

HTML can be absent when report generation is disabled. A Git commit alone is insufficient when source has uncommitted edits; frozen source and hashes are more direct evidence. Back up the complete archive rather than only screenshots or a PDF.

9.2 Rerun and migrate

Within the same environment:

```
sh runs/RUN_ID/reproduce.sh
```

This creates a new run. For a different environment, create a fresh virtual environment, install that archive's `requirements.snapshot.txt` and `code.snapshot/`, then use its reproduction command. Moving archives can invalidate absolute paths. Preserve originals, save a separate configuration with corrected paths and document the changes. Do not substitute today's engine for historical source without disclosure.

Identical inputs, rules and environments should preserve economic results; cross-platform floating-point behavior need not produce byte-identical files. Compare dates, quantities, fees, daily equity and metrics, not just terminal return.

9.3 The existing independent audit

```
python scripts/audit_weekly_relative_reversal.py runs/RUN_ID
```

This script independently recomputes signals, cash/share arithmetic and daily equity for the existing weekly reversal without importing its strategy or engine implementation. It assumes that rule and its cost conventions. It is not a universal audit tool; a changed rule requires a correspondingly independent calculation.

10 Using the standalone research studies

Start with `researchProject/RESEARCH_INDEX.md`. The current studies ask:

1. `01_aapl_tsla_rebalancing/`: how target weights and calendar rebalancing frequency change AAPL/TSLA results.
2. `02_threshold_partial_rebalancing/`: how threshold triggers and partial corrections affect trading, tracking and performance.
3. `03_time_and_asset_robustness/`: whether results persist across entry dates, holding horizons and asset pairs.
4. `04_rebalancing_mechanisms/`: how diversification growth differs from net outperformance of buy and hold.

Follow each study's README and audit instructions. Read its report, protocol, data dictionary, results and code rather than trying to choose its report file in the App. Later studies contain separate calculation engines; similar names do not establish identical implementation details.

Study 04 explicitly separates frictionless close-to-close identities, synthetic experiments and the original opening-execution convention. Mathematical decomposition terms are not substitutes for net opening-execution performance. Fixed asset selection, overlapping windows and parameter comparisons do not automatically establish out-of-sample evidence. Promoting a research idea to a supported strategy requires a precise rule, strategy/factor contracts, tests and verification against the main engine.

11 Maintenance, verification and delivery

11.1 A disciplined change process

1. Record the purpose and expected behavior; identify whether the change concerns data, strategy, engine or UI.
2. Preserve the old configuration and a reference run; version source and do not overwrite historical data files.
3. Implement the smallest necessary change. Material engine changes require tests that detect meaningful failures, including preserved no-look-ahead checks.
4. Run focused tests and the full suite; perform a representative offline end-to-end run when appropriate.
5. For trading-semantic changes, compare orders, fills, equity and costs and explain why results changed.
6. Update documentation/configuration notes; rebuild and verify the native App after interface changes.

11.2 Routine verification commands

```
python -m pytest -q
python -m pip check
# Desktop checks, from desktop/:
npm ci
npm run build
cargo check --manifest-path src-tauri/Cargo.toml
npm run bundle
```

The full Python suite was rerun for this manual on 22 October 2026: **136 tests passed**. This is a dated check, not a guarantee about future edits or a substitute for native release QA. Coverage includes accounting, costs, missing data, contracts, no-look-ahead behavior, configuration, archive reproduction and desktop bridging.

`scripts/check_clean_install.py` verifies wheel installation, tests and an offline example in a temporary environment. It requires a built project wheel under `dist/` and all necessary platform dependencies under `.wheelhouse/`. Missing preparation is not evidence of a strategy failure.

11.3 Dependencies, backups and publication

Upgrade dependencies in a fresh environment, respect `pyproject.toml` ranges, update appropriate lock files and run regression checks. The frontend uses `package-lock.json`; Rust uses `Cargo.lock`. Check native interface compatibility rather than upgrading one side in isolation.

Back up source, configurations, original data, complete important run archives and research protocols. Environments/build caches are reconstructible, but a fully offline recovery also needs compatible wheels and installation tools. Ensure no job is running before cleanup. `runs/` and `.quantlab-desktop/` have different roles.

On 22 October 2026, the owner authorized publishing the current simple strategies and factor. Release 2026.10.22 includes the application, desktop source, configurations, tests and manuals. Independent `researchProject/` studies, real market data, run archives, environments and caches are excluded. The existing GitHub repository was public when checked for this release; its visibility is retained. Inspect the export list and history: `.gitignore` does not remove previously committed material. See `PROJECT_SCOPE.md` for the release boundary.

11.4 Maintaining this manual

Each edition is a self-contained LaTeX source with no external image dependencies. The Chinese edition uses XeLaTeX with system Chinese fonts or Fandol fallbacks; the English edition uses TeX Gyre fonts. Compile twice after changes to resolve the contents and references. Check for overflow, missing glyphs and broken code. Put auxiliary output in a temporary directory; retain only the source and PDF in the final cn/en directories.

12 Troubleshooting and operating checklist

Symptom	First checks
Command missing / import fails	Working directory, activated environment, Python version, editable install and `quantlab--help`.
App cannot connect	Correct project root, working `.venv/bin/python`, moved paths and complete dependencies.
Invalid configuration	Unknown/duplicate YAML keys, date order, parameter types and YAML-relative paths.
Strategy validation fails	Metadata, base class, factory, returned weights, prohibited imports, factor bindings and sizing mode.
Factor module missing	The `factor_paths` module name must match the import and bind the intended source.
No fills	No target, insufficient warmup, inactive dates, inadequate cash, or cancelled missing-open/zero-volume orders. Inspect decisions and orders.
Weekly execution a day late	Confusion between weekly close-time eligibility and Monday-open execution; inspect the next-session date.
Weights not exactly equal	Integer quantities, costs, opening gaps and cash constraints; distinguish targets from realized holdings.
Unchanged curve	A historical run may still be displayed; confirm a new successful identifier after edits.
Save conflict	External source changed; preserve the draft, reload and merge rather than force-overwrite.
Run fails / report absent	Inspect failure marker, logs and report switch. Directory existence alone does not prove success.
Reproduction differs	Compare data, source, parameters, versions and seed; check relocated paths and use of a newer engine.
Source updated, App unchanged	Rebuild the bundle and open the new copy rather than the previous binary.

Before presenting research, confirm that the rule was fixed in advance, data provenance is clear, information timing is valid, costs are explicit, exceptions are disclosed, controls are appropriate and complete archives are preserved. Report poor performance honestly instead of automatically adjusting thresholds until a favorable backtest appears.

A Source basis and version interpretation

This manual was checked on 22 October 2026 against the local `README.md`, `AGENTS.md`, `pyproject.toml`, configurations, core configuration/CLI modules, strategy base/context, factor functions, performance implementation, desktop bridge, `desktop/README.md`, research index and utility scripts. The application version is `2026.10.22`, identified by Git tag `v2026.10.22`. Resolve that tag for the exact release commit. Review scope and evidence are recorded in `AUDIT_2026.10.22.md`.

Implementation takes precedence over stale wording. The older README's V2 roadmap and "single official example" wording describe an earlier stage; approved additional strategies, the desktop app and research studies now exist. This manual does not infer unimplemented live trading, generic automatic optimization, a complete factor-evaluation platform or a self-contained cross-machine installer from those additions.